Competitive Programming Handbook

Personal templates and algorithms

Contents

Frameworks

C++ single-case scaffold

SingleCaseFramework.cpp

Strategy: Fast I/O, common macros (ll, pii, all, loop), one solve() call.

Reuse: Problem with exactly one test case.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void solve() {

}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

C++ multi-case scaffold

MultipleCasesFramework.cpp

Strategy: Same as single-case; reads t and loops solve().

Reuse: T test cases follow style problems.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void solve() {

}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int tc;
    cin >> tc;
    while(tc--) solve();
}
```

Python scaffold

template.py

Strategy: Buffered I/O, raised recursion limit, input helpers, optional input.txt.

Reuse: Any Python problem, single or multi-test.

```python
import os
import sys

# 1. OPTIMIZATIONS
# Increase recursion depth for deep DFS/trees
sys.setrecursionlimit(2000000)

# Fast I/O configuration
input = sys.stdin.readline
def print(*args, sep=" ", end="\n"):
    sys.stdout.write(sep.join(map(str, args)) + end)

# 2. INPUT PARSING SHORTHANDS
def IN_INT(): return int(input())
def IN_STR(): return input().rstrip()
def IN_ARR_INT(): return list(map(int, input().split()))
def IN_ARR_STR(): return input().split()
def IN_MAP_INT(): return map(int, input().split())

# 3. CORE LOGIC
def solve(tc: int):
    """
    Write your problem-solving logic here.
    tc: The current test case number (1-indexed).
    """
    # Example input retrieval:
    # n, m = IN_MAP_INT()
    # arr = IN_ARR_INT()

    # Your logic goes here
    pass

# 4. EXECUTION FRAMEWORK & LOCAL TESTING
def main():
    # Toggle local file I/O if running on a local machine
    # Looks for an 'input.txt' file in the execution folder
    if os.path.exists('input.txt') and 'ONLINE_JUDGE' not in os.environ:
        sys.stdin = open('input.txt', 'r')
        # Uncomment below if you also want to redirect output to a file
        # sys.stdout = open('output.txt', 'w')

    try:
        # Multi-test case handling
        # Change to `t = 1` if the problem has a fixed single testcase
        t = int(input())
    except (ValueError, TypeError):
        t = 1
```

```
    for tc in range(1, t + 1):
        solve(tc)

if __name__ == '__main__':
    main()
```

Search

Binary search on answer

bin_search/bin_search.cpp

Strategy: BS over answer with monotone feasibility predicate try(t).

Reuse: Min/max value with monotone check (time, capacity, threshold).

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

#define P_MAX 1000000000000000000LL

ll n_athletes, pizzas_to_deliver;

ll athletes[200000];

// Returns the amount of pizas the team can carry on a given time.
bool try_deliver_pizzas_in_time(ll tgt_time) {
    ll tot_delivered_pizzas = 0;

    for(int i = 0; i < n_athletes; i++) {
        tot_delivered_pizzas += tgt_time/athletes[i] ;
        if (tot_delivered_pizzas >= pizzas_to_deliver) return true;
    }

    return tot_delivered_pizzas >= pizzas_to_deliver;
}

ll bs(ll lower_bound_time = 1, ll upper_bound_time = P_MAX) {
    ll minimum_time_to_deliver = -1, time_to_deliver;

    while(lower_bound_time <= upper_bound_time) {
        time_to_deliver = (lower_bound_time + upper_bound_time) / 2;
        if(try_deliver_pizzas_in_time(time_to_deliver)){
            upper_bound_time = time_to_deliver -1;
            minimum_time_to_deliver = time_to_deliver;
        } else {
            lower_bound_time = time_to_deliver + 1;
        }
    }

    return minimum_time_to_deliver;
}

void solve() {
    cin >> n_athletes >> pizzas_to_deliver;
    for (int i = 0; i < n_athletes; i++) cin >> athletes[i];

    cout << bs() << endl;
}


int main(void) {
    cin.tie(NULL);
    ios::sync_with_stdio(false);
    solve();
}
```

Ternary search (floating)

EasyProblems/tern_search.cpp

Strategy: Split [l,r] in thirds, discard worse third, epsilon-terminated.

Reuse: Extremum of strictly unimodal continuous function.

Sorting

Merge sort

Sorting/Merge.cpp

Strategy: Divide-and-conquer, O(n log n), stable.

Reuse: Reference; base for counting inversions.

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

int main(void) {
    vector<int> arr = {38, 27, 43, 10, 12, 1, 16, 187};
    int n = arr.size();

    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < arr.size(); i++) cout << arr[i] << " ";
    cout << "\n";

    return 0;
}
```

Counting sort

Sorting/Counting.cpp

Strategy: Bucket histogram, O(n + max).

Reuse: Small non-negative integer range.

```cpp
#include <bits/stdc++.h>
using namespace std;

// Works only for non-negative integers.
void countingSort(vector<int>& v) {
    int n = v.size();
    vector<int> bk(*max_element(v.begin(), v.end()) + 1, 0);

    for (int i : v) {
        bk[i]++;
    }

    int j = 0;
    for (int i = 0; i < bk.size(); i++) {
        while(bk[i]--) {
            v[j] = i;
            j++;
        }
    }
}
```

```cpp
void printVector(vector<int>& v) {
    for (auto i : v)
        cout << i << " ";
    cout << "\n";
}

int main(void) {
    vector<int> v = {1, 3, 6, 9, 9, 3, 5, 9};

    cout << "Before sort: ";
    printVector(v);

    countingSort(v);

    cout << "After sort: ";
    printVector(v);

    cout << "\n";
}
```

Bubble sort

Sorting/Bubble.cpp

Strategy: Repeated adjacent swaps, O(n^2).

Reuse: Tiny n or pedagogical only.

```cpp
#include <bits/stdc++.h>
using namespace std;

void bubble(vector<int>& v) {
    int n = v.size();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (v[j] > v[j + 1]) {
                swap(v[j], v[j + 1]);
            }
        }
    }
}

int main(void) {
    vector<int> v = {5, 1, 4, 2, 8};

    bubble(v);
    for (auto i : v)
        cout << i << " ";
    cout << "\n";

    return 0;
}
```

Distinct numbers

Sorting/DistinctNumbers.cpp

Strategy: Sort then single pass counting first occurrence.

Reuse: Count distinct without hash structures.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()

void solve() {
    int n, x, c = 0;
    vector<int> v;

    cin >> n;

    while (n--) {
        cin >> x;
        v.push_back(x);
    }

    sort(all(v));

    x = -1;
    for (int i : v) {
        if (i != x) {
            x = i;
            c++;
        }
    }

    cout << c << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Two Pointers

Sum of two values

two_pointers/sum_of_two_values.cpp

Strategy: Sort with indices, two-pointer sweep from both ends toward target.

Reuse: Pair with sum = k; problems needing original indices.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void solve() {
    int n, k;
    cin >> n >> k;
    vector<pii> nums(n);
    for(int i = 0; i < n; i++) {
        cin >> nums[i].first;
        nums[i].second = i +1;
    }

    sort(all(nums));

    int l = 0, r = n - 1;

    while (r > l) {
        if(nums[l].first + nums[r].first == k) {
            cout << nums[l].second << " " << nums[r].second << "\n";
            return;
        }
        if (nums[l].first + nums[r].first > k)
            r--;
        else
            l++;
    }

    cout << "IMPOSSIBLE\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    solve();
}
```

Black and white stripes (fixed window)

two_pointers/black_and_white_stripes.cpp

Strategy: Fixed-size sliding window k, update count on entry/exit.

Reuse: Fixed-length substring/subarray min/max count.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void solve() {
    int n, k;
    cin >> n >> k;

    string s;
    cin >> s;

    int ans = 0, l = 0, r = 0;
    for(r = 0; r < k; r++) {
        if (s[r] == 'W') ans++;
    }

    int cont = ans;
    while (r < n) {
        if(s[l] == 'W') cont--;
        if(s[r] == 'W') cont++;
        ans = min(ans, cont);
        l++;
        r++;
    }

    cout << ans << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    int t;
    cin >> t;
    while(t--)
        solve();
```

```
}
```

Bitmasks

Apple division (subset partition)

bitmasks/apple_division.cpp

Strategy: Enumerate all 2^n subsets, minimize partition diff.

Reuse: Subset/partition with n <= ~20.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void solve() {
   int n;
   cin >> n;

    vector<ll> apples(n);
    for(int i = 0; i < n; i++) cin >> apples[i];

    ll ans = INT_MAX;
    for(int mask = 0; mask < (1<<n); mask++) {
        ll g0 = 0, g1 = 0;
        for (int i = 0; i < n; i++) {
            if(mask & (1<<i)) {
                g1 += apples[i];
            } else {
                g0 += apples[i];
            }
        }
        ans = min(ans, abs(g1 - g0));
    }

    cout << ans << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Hamiltonian flights (bitmask DP)

bitmasks/hamiltonian_flights_rec.cpp

Strategy: DP over (current node, visited mask), memoized, mod 1e9+7.

Reuse: TSP-style path counts/costs, n <= ~20.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)
#define MAXN (int)(1e6 + 10)
#define MOD (int)(1e9 + 7)

int n, m, a, b, routes[25][MAXN];
vector<int> graph[MAXN];

int tsp(int cur, int visited_mask) {
    // __builtin_popcount counts how many bits are set in the mask.
    // -> Everyone is already visited?
    if (__builtin_popcount(visited_mask) == n) return (cur == n-1);
    if (routes[cur][visited_mask] != -1) return routes[cur][visited_mask
      ↪ ];

    int ans = 0;
    for (int ngb : graph[cur]) {
        // If ngb is visited, go to the next one.
        if (visited_mask&(1<<ngb)) continue;
        // Else, add the value of that node to the cost memo.
        // By making the OR operation with the visited mask, we will
      ↪ insert it into the mask (marking it as visited).
        else ans = (ans + tsp(ngb, visited_mask|(1<<ngb))) % MOD;
    }

    // Return the number of paths to the current node.
    return routes[cur][visited_mask] = ans;
}

void solve() {
    cin >> n >> m;

    while(m--) {
        cin >> a >> b;
        graph[a-1].push_back(b-1);
    }

    memset(routes, -1, sizeof routes);

    cout << tsp(0, 1) << '\n';
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Trees

Tree diameter

trees/tree_biggest_diameter.cpp

Strategy: DFS tracking longest depth per node; diameter = sum of two best child depths.

Reuse: Diameter / longest path in unweighted tree, single traversal.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

const int maxn = 2e5 + 5;

int n, dp[maxn], diam;
vector<int> graph[maxn];

void dfs(int src, int parent) {
    dp[src] = 0;
    for(auto ngb : graph[src]) {
        if (ngb == parent) continue;
        dfs(ngb, src);
        diam = max(diam, dp[ngb] + dp[src] + 1);
        dp[src] = max(dp[src], dp[ngb] + 1);
    }
}

void solve() {
    cin >> n;
    for (int i = 1; i < n; i++) {
        int node, ngb;
        cin >> node >> ngb;
        graph[node].push_back(ngb);
        graph[ngb].push_back(node);
    }

    dfs(1, 1);

    cout << diam << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Huffman code cost

trees/huffman_code.cpp

Strategy: Greedy multiset merge of two smallest frequencies; accumulate cost.

Reuse: Optimal prefix code, connect ropes, merge stones.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

// The huffman code generates a prefix free encoding language.
// The strategy is to take the two letters that appear the less and put
    ↪ them as leafs.
// Then go summing up until the root (the most common symbol).
ll huffman_cost(vector<int> num_aparicoes) {
    multiset<int> S;
    for(int num : num_aparicoes) {
        S.insert(num);
    }

    ll cost = 0;
    while(S.size() > 1) {
        int leaf1 = *S.begin();
        S.erase(S.find(leaf1));
        int leaf2 = *S.begin();
```

```
        S.erase(S.find(leaf2));
        cost += leaf1 + leaf2;
        S.insert(leaf1 + leaf2);
    }

    return cost;
}

void solve() {

}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Euler tour (tin/tout)

trees/euler_tree.cpp

Strategy: DFS timestamps; subtree becomes contiguous interval.

Reuse: Combine with segtree/BIT for subtree updates and queries.

```
#include <bits/stdc++.h>
using namespace std;

const int maxn = 2e5 + 5;

int tin[maxn], tout[maxn];
int timer;
vector<int> graph[maxn];
vector<int> vec;
void dfs(int v, int p) {
    tin[v] = ++timer;

    for(auto a : graph[v]) {
        if(a == p) continue;
        dfs(a, v);
    }

    tout[v] = ++timer;
    vec.push_back(v);
}
```

Subtree sizes

trees/subordinates.cpp

Strategy: Post-order DFS; each node's size = sum of (child + 1).

Reuse: Subtree size or subtree-aggregate DP.

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

void dfs(vector<vector<int>> &tree, int emp, int sup, vector<int> &
    ↪ subordinates) {
    for(int sub : tree[emp]) {
        dfs(tree, sub, emp, subordinates);
        subordinates[emp] += (subordinates[sub]+1);
    }
}

void solve() {
    int n;
    cin >> n;
    vector<vector<int>> tree(n+1);

    vector<int> subordinates(n+1);
    int dir_man;
    for(int emp = 2; emp <= n; emp++) {
        cin >> dir_man;
        tree[dir_man].push_back(emp);
    }

    dfs(tree, 1, 1, subordinates);

    for(int i = 1 ; i <= n; i++) cout << subordinates[i] << ' ';
    cout << endl;
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Sum of distances (rerooting)

trees/tree_distances.cpp

Strategy: First DFS computes answer from root; second reroots in O(n).

Reuse: Answer for every possible root.

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

int n;
vector<vector<int>> graph;
vector<int> subtree_size;
vector<ll> ans;

void calc_subtrees_sizes_from(int src, int parent, int depth) {
    subtree_size[src] = 1;
    ans[1] += depth;

    for(int ngb : graph[src]) {
        if (ngb != parent) {
           calc_subtrees_sizes_from(ngb, src, depth + 1);
           subtree_size[src] += subtree_size[ngb];
        }
    }
}

void reroot(int src, int parent) {
    for(int ngb : graph[src]) {
        if (ngb != parent) {
            // Nós da sub-árvore de 'v' ficam mais perto: -subtree_size[v
    ↪ ].
            // O resto da árvore fica mais longe: + (n - subtree_size[v])
    ↪ .
            ans[ngb] = ans[src] + n - 2LL * subtree_size[ngb];
            reroot(ngb, src);
        }
    }
}

void solve() {
    cin >> n;
    graph.assign(n + 1, vector<int>());
    ans.assign(n + 1, 0);
    subtree_size.assign(n + 1, 0);
    for (int i = 0; i < n-1; i++) {
        int node, ngb;
        cin >> node >> ngb;
        graph[node].push_back(ngb);
        graph[ngb].push_back(node);
    }

    calc_subtrees_sizes_from(1, -1, 0);

    reroot(1, -1);

    for(int i = 1; i <= n; i++) cout << ans[i] << ' ';
    cout << endl;
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

BST traversals

lpc_training/trees_for_cp/1195_bst.cpp

Strategy: Recursive insert; pre/in/post-order print.

Reuse: BST basics and traversal I/O.

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)

typedef struct Tree {
    int val;
    struct Tree *l, *r;
} Tree;

Tree* insertBst(Tree *r, int node) {
    if (r == NULL) {
        r = (Tree*)malloc(sizeof(Tree));
        r->val = node;
        r->l = NULL;
```

```
        r->r = NULL;
    } else {
        if (node < r->val) {
            r->l = insertBst(r->l, node);
        } else {
            r->r = insertBst(r->r, node);
        }
    }

    return r;
}

void prefix(Tree *r, bool &first) {
    if (r != NULL) {
        if (!first) cout << " ";
        cout << r->val;
        first = false;
        prefix(r->l, first);
        prefix(r->r, first);
    }
}

void infix(Tree *r, bool &first) {
    if (r != NULL) {
        infix(r->l, first);
        if (!first) cout << " ";
        cout << r->val;
        first = false;
        infix(r->r, first);
    }
}

void sufix(Tree *r, bool &first) {
    if (r != NULL) {
        sufix(r->l, first);
        sufix(r->r, first);
        if (!first) cout << " ";
        cout << r->val;
        first = false;
    }
}

void solve(int tc) {
    int n, v;
    cin >> n;
    Tree *bst = NULL;
    while(n--) {
        cin >> v;
        bst = insertBst(bst, v);
    }

    cout << "Case " << tc << ":\n";
    bool first;
    cout << "Pre.: "; first = true; prefix(bst, first); cout << "\n";
    cout << "In..: "; first = true; infix(bst, first); cout << "\n";
    cout << "Post: "; first = true; sufix(bst, first); cout << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int tc;
    cin >> tc;
    for(int i = 1; i <= tc; i++) {
        solve(i);
        cout << "\n";
    }
    return 0;
}
```

Graphs

Dijkstra

graphs/dijkstra.cpp

Strategy: Min-heap SSSP with lazy visited flag.

Reuse: SSSP with non-negative weights, O((V+E) log V).

```
#include <bits/stdc++.h>
using namespace std;

#define MAX 1000
#define pii pair<int,int>

int dist[MAX], nodes, edges;
bool visited[MAX];

// node -> list of {ngb, weight}.
vector<pii> graph[MAX];

// dist[node], node
priority_queue<pii, vector<pii>, greater<pii>> pq;

void dijkstra(int src) {
    for(int i = 1; i <= nodes; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }

    dist[src] = 0;
    pq.push({0, src});

    while(!pq.empty()) {
        auto [_, cur] = pq.top();
        pq.pop();

        if(visited[cur]) continue;
        visited[cur] = true;

        for(auto[ngb, weight] : graph[cur]) {
            if(visited[cur]) continue;
            if(dist[ngb] > dist[cur] + weight) {
                dist[ngb] = dist[cur] + weight;
                pq.push({dist[ngb], ngb});
            }
        }
    }
}
```

Floyd-Warshall

graphs/floyd_warshall.cpp

Strategy: Triple loop over intermediate k; O(V^3).

Reuse: All-pairs shortest paths, V <= ~400, handles negative edges.

```
// Used to find the minimum distance between all pair of nodes.
#include <bits/stdc++.h>
using namespace std;
#define MAX 1000

int graph[MAX][MAX], dist[MAX][MAX];


void floyd_warshall(int n, int m) {
    for(int i = 1; i<= n; i++) {
        for(int j = 1; j<= n; j++) {
            if(i == j) dist[i][j] = 0;
            else dist[i][j] = INT_MAX;
        }
    }

    while (m--) {
        int a, b;
        int c;
        cin >> a >> b >> c;

        dist[a][b] = min(dist[a][b], c);
        dist[b][a] = min(dist[b][a], c);
    }

    for(int k = 1; k <= n; k++) {
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j<= n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}
```

BFS distance

graphs/bfs_dist.cpp

Strategy: Queue-based shortest hop distance in unweighted graph.

Reuse: Unweighted SSSP, level-order, grid distance.

```
#include <bits/stdc++.h>
using namespace std;

#define MAX 1000
int dist[MAX];
bool visited[MAX];
vector<int> graph[MAX];
queue<int> q;

void bfs(int src) {
    dist[src] = 0;
    visited[src] = true;
    q.push(src);

    while(!q.empty()) {
        int cur = q.front();
        q.pop();
        visited[cur] = true;

        for(int ngb : graph[cur]) {
            if (!visited[ngb]) {
                dist[ngb] = dist[cur] + 1;
                q.push(ngb);
            }
        }
    }
}
```

Bipartite check (2-coloring)

graphs/beecrowd/separate_rooms_1979.cpp

Strategy: BFS 2-color per component; same-colored neighbor = odd cycle.

Reuse: Bipartiteness / can-we-split-in-two.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)
#define adjacencyMap(type) map<type, set<type>>
#define visitedMap(type) map<type, bool>

void readAndFillGraph(int tc, adjacencyMap(int) *graph) {
    for (int i = 1; i <= tc; i++) (*graph)[i] = {};
    while(tc--) {
        int node;
        cin >> node;
        cin.ignore();
        string line;
        set<int> nodeneighbours;
        if(getline(cin, line)) {
            istringstream iss(line);
            int neighbour;

            while(iss >> neighbour) {
                nodeneighbours.insert(neighbour);
            }
        }

        (*graph)[node] = nodeneighbours;
        for(int neighbour : nodeneighbours) {
            (*graph)[neighbour].insert(node);
        }

        nodeneighbours.clear();
    }
}

// initVisited maps all nodes as unvisited.
visitedMap(int) initVisited(adjacencyMap(int) graph) {
    visitedMap(int) visited;
    for (auto const&[node, neighbous] : graph) {
        visited[node] = false;
    }
    return visited;
}

bool isBipartite(adjacencyMap(int) graph) {
    visitedMap(int) visited = initVisited(graph);
    map<int, bool> color = visited;

    // Choses a node for each component.
    for(auto const&[node, neighbours] : graph) {
        if (visited[node]) continue;

        // Starts a search queue for a specific node.
        queue<int> q; q.push(node);

        // Attribute color 1 to the node,
        color[node] = true;
        visited[node] = true;
        while(!q.empty()) {
            int curNode = q.front(); q.pop();
            for (int neighbour : graph[curNode]) {
                if (visited[neighbour] && color[curNode] == color[
    ↪ neighbour]) {
                    // Found an odd cycle (two neighbours with same
    ↪ color).
                    return false;
                } else if(!visited[neighbour]) {
                    // Inverts the color of the cur node based on the
    ↪ neighbour color.
                    color[neighbour] = !(color[curNode]);
                    visited[neighbour] = true;
                    q.push(neighbour);
                }
            }
        }
    }

    // No odd cycle was found.
    return true;
}

void solve(int tc) {
    adjacencyMap(int) graph;

    readAndFillGraph(tc, &graph);

    if (isBipartite(graph)) {
        cout << "SIM\n";
    } else {
        cout << "NAO\n";
    }
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int tc;
    cin >> tc;
    while(tc != 0){
        solve(tc);
        cin >> tc;
    }
}
```

Connected components

graphs/beecrowd/connected_components_1082.cpp

Strategy: BFS from every unvisited node, collect components.

Reuse: Count/list components in undirected graph.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)
#define adjacencyMap(type) map<type, set<type>>
#define visitedMap(type) map<type, bool>
const string alpha = "abcdefghijklmnopqrstuvwxyz";

// initVisited maps all nodes as unvisited.
visitedMap(char) initVisited(adjacencyMap(char) graph) {
    visitedMap(char) visited;
    for (auto const&[node, neighbous] : graph) {
        visited[node] = false;
    }
    return visited;
}

adjacencyMap(char) buildGraph() {
    int v, e;
    cin >> v >> e;
    adjacencyMap(char) graph; for(int i = 0; i < v; i++) graph[alpha[i]]
     ↪  = {};

    while(e--) {
        char a, b;
        cin >> a >> b;
        graph[a].insert(b);
        graph[b].insert(a);
    }

    return graph;
}

set<char> bfs(adjacencyMap(char) graph, char startNode, visitedMap(char)
     ↪  *visited) {
    set<char> component;
    queue<char> q;
    q.push(startNode);
    (*visited)[startNode] = true;
    component.insert(startNode);

    while(!q.empty()) {
        char curNode = q.front(); q.pop();
        for(char neighbour : graph[curNode]) {
            if((*visited)[neighbour]) continue;

            q.push(neighbour);
            (*visited)[neighbour] = true;
            component.insert(neighbour);
        }
    }

    return component;
}

void solve(int tc) {
    adjacencyMap(char) graph = buildGraph();
    visitedMap(char) visited = initVisited(graph);
    set<set<char>> components;

    for(auto const&[node, _] : graph) {
        if (!visited[node]) {
            set<char> component = bfs(graph, node, &visited);
            components.insert(component);
        }
    }

    cout << "Case #" << tc << ":\n";
    for(auto component : components) {
        for (char node : component) {
            cout << node << ",";
        }
        cout << "\n";
    }
    cout << components.size() << " connected components\n\n";
}

int main(void) {
    ios::sync_with_stdio(false);
```

```
    cin.tie(NULL);

    int tc, i = 1;
    cin >> tc;
    while(tc--) {
        solve(i);
        i++;
    }
}
```

Dynamic Programming

Kadane (max subarray sum)

Dynamic_Programming/MaxSubarraySum.cpp

Strategy: Running sum reset when extending is worse than restarting.

Reuse: Max contiguous subarray sum, O(n).

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()

void solve() {
    vector<int> v;
    int x, n = 0;
    cin >> n;
    int c = n;

    while (n--) {
        cin >> x;
        v.push_back(x);
    }

    int best = 0, sum = 0;
    for (int i = 0; i < c; i++) {
        sum = max(v[i], sum + v[i]);
        best = max(sum, best);
    }

    cout << best << "\n";
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Diagonal path count (memoized)

Dynamic_Programming/DiagonalPathCount.cpp

Strategy: Top-down recursion with map memo; grid paths = binomial coeff.

Reuse: Template for 2-parameter memoized DP with map cache.

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array

map<pair<int, int>, ll> memo;

ll countDiagonalPaths(int rows, int columns) {
    pair<int, int> p = make_pair(rows, columns);

    if (memo.count(p)) {
        return memo[p];
    }

    if (rows == 1 || columns == 1) {
        return 1;
    }

    ll res = countDiagonalPaths(rows - 1, columns) + countDiagonalPaths(
      ↪ rows, columns - 1);
    memo[p] = res;
    return res;
}

int main() {
    int rows, columns;
    cin >> rows >> columns;

    cout << countDiagonalPaths(rows, columns) << "\n";
}
```

Segment Tree

Dynamic range sum queries

seg_tree/dynamic_range_sum_queries.cpp

Strategy: Recursive segtree: point-assign update, range-sum query, O(log n). VERIFY update sibling-index before reuse.

Reuse: Point update + range sum; use Fenwick if merge is just sum.

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)
#define MAXN 2000010

ll  seg[4*MAXN];
ll  vet[MAXN];

ll build(int node, int l, int r) {
    if (l == r) return seg[node] = vet[l];
    int mid = (l + r) / 2;
    seg[node] = build(2*node, l, mid) + build(2*node+1, mid+1, r);
    return seg[node];
}

ll update(int node, int l, int r, int k, int u) {
if(l == r) return seg[node] = u;
    int mid = (l + r) / 2;
    if (k <= mid) {
        seg[node] = seg[2*node+1] + update(2*node, l, mid, k, u);
    } else if (k > mid) {
        seg[node] = seg[2*node] + update(2*node + 1, mid+1, r, k, u);
    }

    return seg[node];
}

ll sum(int node, int l, int r, int l0, int r0) {
    if (l >= l0 && r <= r0)
        return seg[node];
    else if(l > r0 || r < l0)
        return 0;
    int mid = (l + r) / 2;
    return sum(2*node, l, mid, l0, r0) + sum(2*node+1, mid+1, r, l0, r0)
      ↪ ;
}

void solve() {
    ll n, q;
    cin >> n >> q;

    for(int i = 1; i <= n; i++) cin >> vet[i];
    build(1, 1, n);

    while(q--) {
        int op;
        cin >> op;

        switch(op) {
            case 1:
                ll k, u;
                cin >> k >> u;
                update(1, 1, n, k, u);
                break;
            case 2:
                ll l0, r0;
                cin >> l0 >> r0;
                cout << sum(1, 1, n, l0, r0) << "\n";
                break;
        }
    }
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    solve();
}
```

Stable Matching

Gale-Shapley (C++)

StableMatching/gale_shapley.cpp

Strategy: Proposers propose in preference order; receivers keep best via rank map.

Reuse: Any stable matching / preference pairing.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ar array
#define pb push_back
#define mp make_pair
#define pii pair<int, int>
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define loop(it, start, end) for(int it = start; it < end; it++)
#define VERBOSE false

map<string, string> gs(map<string, vector<string>> propPrefs, map<string
    ↪ , vector<string>> recPrefs) {
    // Precompute each receivers rank of proposers for comparisons.
    map<string, map<string, int>> recRank;
    for (auto const& [rec, propRank] : recPrefs) {
        for(int i = 0; i < propRank.size(); i++) {
            recRank[rec][propRank[i]] = i;
        }
    }

    // All proposers start free.
    deque<string> freeProps;

    // Store the index where each proposer stop in the last proposal.
    map<string, int> nextProposerIndex;
    for (auto const& [prop, _] : propPrefs) {
        freeProps.push_back(prop);
        nextProposerIndex.insert(mp(prop, 0));
    }

    // Final output matches.
    map<string, string> matches;

    // While there are proposers that are free.
    while(!freeProps.empty()) {
        // Remove the first proposer.
        string proposer = freeProps.front(); freeProps.pop_front();

        // If proposer has already proposed to every receiver, skip (no
    ↪ more matches possible).
        if (nextProposerIndex[proposer] >= propPrefs[proposer].size()) {
            if(VERBOSE) cout << proposer << " has already proposed to
    ↪ everyone\n";
            continue;
        }

        // Highest-ranked receiver proposer hasn't proposed to yet.
        string receiver = propPrefs[proposer][nextProposerIndex[proposer
    ↪ ]];
        nextProposerIndex[proposer]++;
        if(VERBOSE) cout << proposer << " proposes to " << receiver << "
    ↪ \n";

        // If receiver is free.
        if (matches.find(receiver) == matches.end()) {
            // (proposer, receiver) become engaged.
            matches.insert(make_pair(receiver, proposer));
            if(VERBOSE) {
                cout << receiver << " is free.\n";
                cout << proposer << " and " << receiver << " become
    ↪ engaged!\n";
            }
        } else {
            string currentReceiverPair = matches[receiver];

            // If receiver prefers current pair over new proposer.
            if (recRank[receiver][currentReceiverPair] < recRank[
    ↪ receiver][proposer]) {
                // Proposer remains free; try next receiver later.
                freeProps.push_back(proposer);
                if(VERBOSE) cout << proposer << " remains free.\n";
            } else {
                // Receiver prefers new proposer over current pair.
                matches[receiver] = proposer;

                // The current pair becomes free.
                freeProps.push_back(currentReceiverPair);

                if(VERBOSE) cout << receiver << " and " << proposer << "
    ↪  engaged. "  << currentReceiverPair << " becomes free.\n";
            }
        }
        // If proposer still hasn't proposed to everyone and is free. It
    ↪ re-enter loop
        // through the deque. Loop continues while there exists a free
    ↪ proposer who
        // hasn't proposed to all receivers.
    }

    return matches;
}

int main(void) {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    map<string, vector<string>> propPrefs = {
        {"Atlanta", {"Wayne", "Val", "Yolanda", "Zeus", "Xavier"}},
        {"Boston", {"Yolanda", "Wayne", "Val", "Xavier", "Zeus"}},
        {"Chicago", {"Wayne", "Zeus", "Xavier", "Yolanda", "Val"}},
        {"Detroit", {"Val", "Yolanda", "Xavier", "Wayne", "Zeus"}},
        {"El Paso", {"Wayne", "Yolanda", "Val", "Zeus", "Xavier"}}
    };

    map<string, vector<string>> recPrefs = {
        {"Val", {"El Paso", "Atlanta", "Boston", "Detroit", "Chicago"}},
        {"Wayne", {"Chicago", "Boston", "Detroit", "Atlanta", "El Paso"
    ↪ }},
        {"Xavier", {"Boston", "Chicago", "Detroit", "El Paso", "Atlanta"
    ↪ }},
        {"Yolanda", {"Atlanta", "El Paso", "Detroit", "Chicago", "Boston
    ↪ "}},
        {"Zeus", {"Detroit", "Boston", "El Paso", "Chicago", "Atlanta"}}
    };

    map<string, string> ans = gs(propPrefs, recPrefs);

    for (auto const& [key, val] : ans) {
        cout << key << " <-> " << val << "\n";
    }
}
```

Gale-Shapley (Python)

StableMatching/gale_shapley_simple.py

Strategy: Compact port of Gale-Shapley using deque of free proposers.

Reuse: Python readability version of the C++ above.

```python
from collections import deque

def gale_shepley(p_prefs: dict[str, list[str]], r_prefs: dict[str, list[
    ↪ str]]) -> set[tuple]:
    r_rank = {r: {p: i for i, p in enumerate(pref)} for r, pref in
    ↪ r_prefs.items()}
    free_p = deque(p_prefs.keys())
    next_idx = {p: 0 for p in p_prefs}
    engaged_to = {}  # receiver -> proposer
    while free_p:
        p = free_p.popleft()
        if next_idx[p] >= len(p_prefs[p]): continue  # proposed to
    ↪ everyone
        r = p_prefs[p][next_idx[p]]
        next_idx[p] += 1
        if r not in engaged_to:
            engaged_to[r] = p
        else:
            p_prime = engaged_to[r]
            if r_rank[r][p_prime] < r_rank[r][p]:
                free_p.append(p)
            else:
                engaged_to[r] = p
                free_p.append(p_prime)
    return {(p, r) for r, p in engaged_to.items()}

if __name__ == "__main__":
    p_prefs: dict[str, list[str]] = {
        "Atlanta": ["Wayne", "Val", "Yolanda", "Zeus", "Xavier"],
        "Boston": ["Yolanda", "Wayne", "Val", "Xavier", "Zeus"],
        "Chicago": ["Wayne", "Zeus", "Xavier", "Yolanda", "Val"],
        "Detroit": ["Val", "Yolanda", "Xavier", "Wayne", "Zeus"],
        "El Paso": ["Wayne", "Yolanda", "Val", "Zeus", "Xavier"]
    }

    r_prefs: dict[str, list[str]] = {
        "Val": ["El Paso", "Atlanta", "Boston", "Detroit", "Chicago"],
        "Wayne": ["Chicago", "Boston", "Detroit", "Atlanta", "El Paso"],
        "Xavier": ["Boston", "Chicago", "Detroit", "El Paso", "Atlanta"
    ↪ ],
        "Yolanda": ["Atlanta", "El Paso", "Detroit", "Chicago", "Boston"
    ↪ ],
        "Zeus": ["Detroit", "Boston", "El Paso", "Chicago", "Atlanta"]
    }

    result: set[tuple] = gale_shepley(p_prefs=p_prefs, r_prefs=r_prefs)

    print(f"Result: {result}")
```

Excluded

- bin_search/tern_search.cpp: empty file (see EasyProblems/tern_search.cpp instead).
- lpc_training/dynamic_prgramming/1838_philosophal_stone.cpp: unfinished stub.